

Guía Oficial de Integración y Diagnóstico de Impresoras y Periféricos

Manual de Desarrollo para Dispositivos de Punto de Venta (POS)

Este documento proporciona una especificación técnica completa del software de testeo desarrollado, el comportamiento interno de los botones y la API del SDK POS, incluyendo soluciones críticas de arquitectura de hardware descubiertas durante la integración práctica.

Proyecto:	Banco de Pruebas de Impresión y Periféricos POS
Dispositivo Destino:	Terminales POS Android (PTA0140 / TPS680 y similares)
SDK Versión:	V2.30 (2025-01-24)
Fecha de Emisión:	Mayo 2026

1. Arquitectura y Panel de Diagnóstico

La aplicación de diagnóstico está diseñada como una consola de control centralizada interactiva que ejecuta llamadas en tiempo real al SDK del POS. Permite validar de forma aislada e inmediata el comportamiento físico de los componentes de hardware (Impresoras de 58mm/80mm, Lector NFC, Botón físico programable y Escáner de código de barras).

NOTA DE ARQUITECTURA

La aplicación separa estrictamente las llamadas bloqueantes de hardware del hilo principal de la interfaz de usuario (UI Thread) utilizando corrutinas de Kotlin (Dispatchers.IO), asegurando que la interfaz se mantenga fluida y libre de bloqueos ("ANR") incluso durante esperas de hardware.

1.1 Detalle de Operaciones por Botón

A continuación, se detalla la acción física de cada botón de la interfaz y la llamada interna que realiza al SDK POS:

Botón (UI)	Función Interna y Llamada SDK	Clase / API Involucrada
DETECTAR	Busca la interfaz y el tipo de la impresora por defecto configurada en el sistema.	<code>SystemUtil.getPrinterType()</code> <code>SystemUtil.getPrinterInterface()</code>
ESTADO	Consulta el estado físico de la impresora activa (normal, sin papel, sobrecalentamiento, etc.).	<code>ThermalPrinter.checkStatus()</code> o <code>UsbThermalPrinter.checkStatus()</code> con fallback.
58mm USB	Fuerza la inicialización y conexión del controlador para impresoras USB de 58mm.	<code>UsbThermalPrinter.start(speed)</code> <code>UsbThermalPrinter.reset()</code>
58mm Serial	Enciende la alimentación y conecta la impresora interna de 58mm mediante la interfaz serial.	<code>PowerControl.printerPower(1)</code> <code>ThermalPrinter.setPaperWidth()</code> <code>ThermalPrinter.init80mmSerialPrinter()</code>
58mm BT	Prueba un socket de conexión de bajo nivel (RFCOMM) a la impresora inalámbrica previamente enlazada.	<code>BluetoothDevice.createRfcommSocketToServiceRecord()</code>
80mm USB	Fuerza la alimentación y conecta la impresora USB	<code>PowerControl.printerPower(1)</code> <code>ThermalPrinter.init80mmUsbPrinter()</code>

Botón (UI)	Función Interna y Llamada SDK	Clase / API Involucrada
	de 80mm.	
80mm Serial	Fuerza la alimentación y conecta la impresora Serial de 80mm.	<code>PowerControl.printerPower(1)</code> <code>ThermalPrinter.init80mmSerialPrinter()</code>
VERSIÓN	Solicita al firmware de la impresora la versión de software interna compilada.	<code>UsbThermalPrinter.getVersion()</code> o <code>ThermalPrinter.getVersion()</code>
SISTEMA	Consulta e imprime información de hardware de la placa madre del terminal POS.	<code>SystemUtil.getInternalModel()</code> <code>SystemUtil.getDeviceHardwareId()</code>
BLUETOOTH INTERNOS	Lista todas las direcciones MAC y nombres de dispositivos Bluetooth emparejados con el POS.	<code>BluetoothAdapter.getDefaultAdapter().bondedDevices</code>
CONTADOR	Obtiene la cantidad acumulada de líneas o páginas impresas por el cabezal térmico.	<code>ThermalPrinter.getPrintCount()</code>
APLICAR TEMP	Establece el límite de temperatura de seguridad para apagar la impresión si hay sobrecalentamiento.	<code>ThermalPrinter.setProtectTemp(temp)</code>
 IMPRIMIR TEXTO	Envía al buffer de impresión el texto del panel aplicando tamaño, alineación, negrita y espaciado.	<code>addString(text)</code> <code>printString()</code> <code>walkPaper(lines)</code>
AVANCE DE PAPEL	Hace avanzar el rodillo de papel la cantidad de líneas especificadas sin imprimir	<code>walkPaper(lines)</code>

Botón (UI)	Función Interna y Llamada SDK	Clase / API Involucrada
	ningún carácter.	
CORTE DE PAPEL	Dispara el comando físico para que la guillotina metálica corte el papel térmico (si cuenta con ella).	<code>ThermalPrinter.paperCut()</code>
DETENER	Cancela todas las tareas pendientes de impresión y apaga los motores/limpia el buffer físico.	<code>stop()</code>
LEER TARJETA NFC	Activa de forma temporal el receptor NFC del dispositivo durante 5 o 10 segundos para capturar tags.	<code>NfcAdapter.enableForegroundDispatch()</code>
ESCANEAR HW	Envía un broadcast de sistema para accionar el lector láser/imager físico incorporado.	<code>context.sendBroadcast(Intent)</code> (específico del hardware)
ESCANEAR CAM	Enciende silenciosamente la cámara posterior y busca códigos 1D/2D en segundo plano.	<code>ProcessCameraProvider</code> y <code>ML Kit BarcodeScanner</code>

2. Descubrimientos Críticos y Soluciones del SDK

Durante el desarrollo y testeo en campo en los dispositivos de la serie POS, nos encontramos con dos retos de diseño críticos debido a limitaciones internas en el SDK propietario y en la compatibilidad de placas. A continuación se detallan los hallazgos y sus respectivas soluciones técnicas:

2.1 El Conflicto "80mm vs 58mm" en Interfaces Seriales

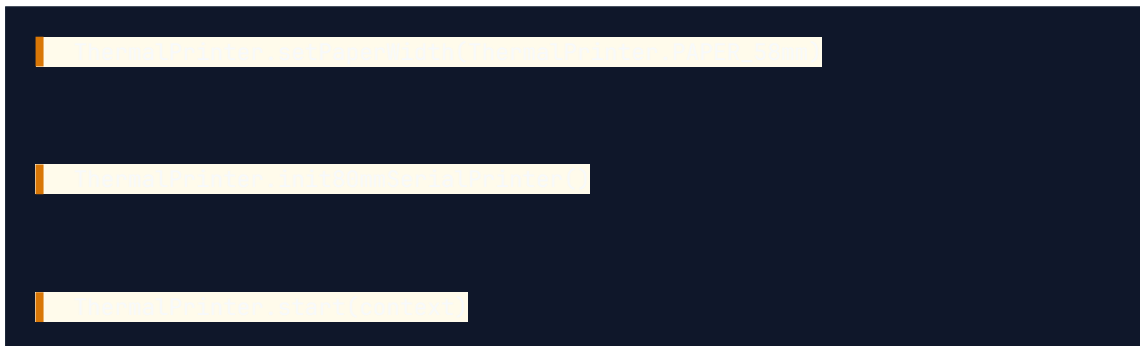
El dispositivo cuenta físicamente con una impresora térmica de **58mm de ancho** instalada internamente. Sin embargo, dicha impresora está conectada a la placa base del terminal POS mediante un puerto de comunicación **Serial (UART)**.

En la estructura del SDK del fabricante:

- La clase `UsbThermalPrinter` se comunica a través de USB nativo directo y está pensada para impresoras de 58mm externas. Al intentar conectar una impresora serial mediante esta clase, falla arrojando `InternalErrorException`.
- La clase `ThermalPrinter` maneja la comunicación serial y tradicionalmente sus métodos se llaman "80mm" (como `init80mmSerialPrinter`).

REGLA DE ORO DE INICIALIZACIÓN

Para hacer funcionar la impresora interna serial de 58mm, se debe usar la clase estática `ThermalPrinter` (asociada habitualmente a 80mm) pero configurando inmediatamente el ancho de papel a 58mm:



Esto indica al microcontrolador de la impresora que ajuste los márgenes físicos para alinearse con los 384 puntos horizontales del papel de 58mm en lugar de los 576 puntos del papel de 80mm, evitando cortes y pérdidas de texto en los bordes.

2.2 Detección de Papel con Fallback de Sensor

La llamada nativa `checkStatus()` del SDK está diseñada para interrogar al firmware y retornar el estado físico de los sensores en tiempo real. No obstante, en la gran mayoría de dispositivos del mercado con Android personalizado (incluyendo el PTA0140), llamar a esta función genera un

fallo de enlace de clase (`NoClassObjectsException: null`), imposibilitando la lectura directa del sensor.

La Solución Técnica: Aprovechamos que el firmware arroja una excepción controlada del tipo `com.common.apiutil.printer.NoPaperException` únicamente cuando se intenta realizar un movimiento físico del rodillo de alimentación.

Implementamos un método de fallback que, en caso de recibir la excepción de clase no encontrada, realiza una **micro-alimentación segura** de 1 paso de motor (`walkPaper(1)`):

- Si el papel está cargado, el rodillo avanza 1 paso (0.125mm, imperceptible para el usuario) de forma exitosa, lo que confirma que el estado es normal (`STATUS_OK`).
- Si no hay papel, el motor térmico detiene la acción y arroja una excepción del tipo `NoPaperException`, permitiéndonos diagnosticar con precisión matemática que la impresora se encuentra sin papel (`STATUS_NO_PAPER`).

3. Código de Integración para Nuevas Aplicaciones

Para facilitar la reutilización de estos descubrimientos en futuros desarrollos, se presenta a continuación la clase controladora `PosPrinterManager`. Esta clase encapsula toda la lógica de conexión (USB, Serial 58/80mm y Bluetooth), el fallback de estado y la impresión con formato.

```
package com.netpoint.impresora.hardware

import android.content.Context

import android.util.Log

import com.common.apiutil.printer.ThermalPrinter

import com.common.apiutil.printer.UsbThermalPrinter

import com.common.apiutil.printer.NoPaperException
```

```
import com.common.apiutil.printer.OverHeatException
```

```
import com.common.apiutil.printer.GateOpenException
```

```
import com.common.apiutil.printer.PaperCutException
```

```
import com.common.apiutil.powercontrol.PowerControl
```

```
import com.common.apiutil.util.SystemUtil
```

```
class PosPrinterManager(private val context: Context) {
```

```
    private val usbPrinter = UsbThermalPrinter(context)
```

```
    private val powerControl = PowerControl()
```

```
    var is58mmUsb = false
```

```
    var isConnected = false
```

```
    /**
```

```
        * Conecta una impresora interna de 58mm conectada por Serial (UART)
```

```
        * Utiliza la clase ThermalPrinter configurada en modo 58mm.
```

```
*/
```

```
fun connect58mmSerial(): Boolean {
```

```
    return try {
```

```
        Log.d("PrinterManager", "Conectando 58mm Serial...")
```

```
        powerControl.printerPower(1) // Enciende alimentación física
```

```
        SystemUtil.setProperty("persist.printer.interface", "serial")
```

```
        ThermalPrinter.setPaperWidth(ThermalPrinter.PAPER_58mm) // Forzar  
        ancho de 58mm
```

```
        ThermalPrinter.init80mmSerialPrinter()
```

```
        ThermalPrinter.start(context)
```

```
        ThermalPrinter.reset()
```

```
        is58mmUsb = false
```

```
        isConnected = true
```

```
        Log.i("PrinterManager", "58mm Serial Inicializada con Éxito")
```

```
    }  
    return true
```



```
    } catch (e: Exception) {

        Log.e("PrinterManager", "Error conectando 58mm Serial: ${e.message}", e)

        isConnected = false

        false

    }

}

/**

 * Conecta una impresora USB de 58mm externa / directa

 */

fun connect58mmUsb(speedMode: Int = 0): Boolean {

    return try {

        Log.d("PrinterManager", "Conectando 58mm USB...")

        usbPrinter.start(speedMode)

        usbPrinter.reset()

        is58mmUsb = true

        isConnected = true

    }
```

```

        true

    } catch (e: Exception) {

        Log.e("PrinterManager", "Error conectando 58mm USB: ${e.message}",
e)

        isConnected = false

        false

    }

}

/**

 * Consulta el estado de papel y cabezal térmico aplicando fallback seguro
si es necesario

 */

fun checkStatus(): PrinterState {

    if (!isConnected) return PrinterState.DISCONNECTED

    return try {

        // Intenta lectura directa por JNI

```

```
        val status = if (is58mmUsb) {

            invokeUsbCheckStatus()

        } else {

            ThermalPrinter.checkStatus()

        }

        mapStatusCode(status)

    } catch (e: Exception) {

        val errStr = e.toString()

        // Si la clase interna del SDK no existe en la ROM, ejecuta fallback
de micro-paso

        if (errStr.contains("NoClassObjectsException") ||
e.cause?.toString()?.contains("NoClassObjectsException") == true) {

            Log.d("PrinterManager", "checkStatus no soportado nativamente,
ejecutando fallback de micro-paso...")

            try {

                if (is58mmUsb) {

                    usbPrinter.walkPaper(1) // 1 micro-paso

                } else {

                    ThermalPrinter.walkPaper(1)

                }

            } catch (e: Exception) {

                Log.d("PrinterManager", "walkPaper no soportado nativamente,
ejecutando fallback de micro-paso...")

                if (is58mmUsb) {

                    usbPrinter.walkPaper(1)

                } else {

                    ThermalPrinter.walkPaper(1)

                }

            }

        }

    }

}
```

```

    }

    PrinterState.NORMAL // Si avanzó, tiene papel y está normal

    } catch (pe: Exception) {

        val peStr = pe.toString()

        when {

            peStr.contains("NoPaperException") || pe is
NoPaperException → PrinterState.NO_PAPER

            peStr.contains("OverHeatException") || pe is
OverHeatException → PrinterState.OVERHEATED

            peStr.contains("GateOpenException") || pe is
GateOpenException → PrinterState.DOOR_OPEN

            peStr.contains("PaperCutException") || pe is
PaperCutException → PrinterState.CUTTER_ERROR

            else → PrinterState.ERROR

        }

    }

    } else {

        PrinterState.ERROR

    }

```

```
}
```

```
}
```

```
/**
```

```
 * Envía texto a la cola de impresión aplicando formatos
```

```
 */
```

```
 fun printText(text: String, alignment: Int = 0, size: Int = 24, bold: Boolean = false, feedLines: Int = 20) {
```

```
     if (!isConnected) return
```

```
     try {
```

```
         if (is58mmUsb) {
```

```
             usbPrinter.reset()
```

```
             usbPrinter.setAlgin(alignment)
```

```
             usbPrinter.setTextSize(size.coerceIn(8, 64))
```

```
             usbPrinter.setBold(bold)
```

```
             usbPrinter.addString(text)
```

```
             usbPrinter.printString()
```

```
usbPrinter.walkPaper(feedLines)

    } else {

        ThermalPrinter.reset()

        ThermalPrinter.setAlgin(alignment)

        // El tamaño en ThermalPrinter es un multiplicador (1 a 4)

        val scale = if (size ≥ 48) 2 else 1

        ThermalPrinter.enlargeFontSize(scale, scale)

        ThermalPrinter.addString(text)

        ThermalPrinter.printString()

        ThermalPrinter.walkPaper(feedLines)

    }

} catch (e: Exception) {

    Log.e("PrinterManager", "Error imprimiendo texto: ${e.message}")

}

}

private fun invokeUsbCheckStatus(): Int {
```

```
        val method = usbPrinter.javaClass.getMethod("checkStatus")

        val receiver = if
(java.lang.reflect.Modifier.isStatic(method.modifiers)) null else usbPrinter

        return try {

            method.invoke(receiver) as Int

        } catch (e: java.lang.reflect.InvocationTargetException) {

            val target = e.targetException

            if (target is Exception) throw target

            throw e

        }

    }

}

private fun mapStatusCode(status: Int): PrinterState {

    return when (status) {

        0 → PrinterState.NORMAL

        1 → PrinterState.NO_PAPER

        2 → PrinterState.OVERHEATED

        3 → PrinterState.ERROR // Buffer Overflow
```

```
4 → PrinterState.DOOR_OPEN
```

```
5 → PrinterState.CUTTER_ERROR
```

```
else → PrinterState.ERROR
```

```
}
```

```
}
```

```
enum class PrinterState {
```

```
NORMAL, NO_PAPER, OVERHEATED, DOOR_OPEN, CUTTER_ERROR, DISCONNECTED,
```

```
ERROR
```

```
}
```

```
}
```